CENTRO UNIVERSITÁRIO UNIFACVEST
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO
DISCIPLINA DE ESTRUTURA DE DADOS
PROFESSOR: Me. MARCELO GOULART SOUZA

ANÁLISE COMPARATIVA DE DESEMPENHO ENTRE OS ALGORITMOS DE ORDENAÇÃO MERGESORT BOTTOM-UP E QUICKSORT CLÁSSICO

Jeferson Gomes Martins
João Vitor Gonçalves de Almeida
Juan Flor
Igor Jesus
Guilherme Silva

RESUMO

Neste trabalho, comparamos o desempenho de dois algoritmos de ordenação por comparação: o Mergesort na versão *bottom-up* (iterativa) e o Quicksort clássico, com pivô no elemento central, ambos implementados em C++. Cada algoritmo ordenou conjuntos de 30, 300, 3.000, 30.000 e 300.000 números inteiros aleatórios, com 20 amostras para cada tamanho, totalizando 200 ordenações. Em cada amostra, os dois algoritmos receberam exatamente os mesmos dados, e conferimos se todos os resultados saíram ordenados. Resumimos os tempos por estatística descritiva (média, mediana, desvio padrão, variância, coeficiente de variação, máximo e mínimo) e os ilustramos com *boxplots* e gráficos de barras. Nos conjuntos de até 3.000 elementos os tempos ficaram na casa dos microssegundos e as diferenças entre os algoritmos (de 1,4% a 4,8%) ficaram dentro da variação das amostras, caracterizando empate técnico. A partir de 30.000 elementos o Quicksort clássico foi mais rápido, com tempo médio 8,9% e 15,2% menor para 30.000 e 300.000 elementos, respectivamente. O crescimento do tempo dos dois algoritmos acompanhou a previsão teórica O(n log n): a cada vez que a entrada foi multiplicada por 10, o tempo médio cresceu entre 11,2 e 15,4 vezes. Testes complementares com entradas já ordenadas, invertidas e com muitos valores repetidos mantiveram a vantagem do Quicksort. Concluímos que o Quicksort clássico é a melhor escolha quando se busca menor tempo médio e pouco uso de memória, enquanto o Mergesort *bottom-up* é preferível quando são exigidos estabilidade e tempo garantido no pior caso.

Palavras-chave: Algoritmos de ordenação. Mergesort. Quicksort. Análise de desempenho. Estatística descritiva.

1 INTRODUÇÃO

Ordenar é dispor um conjunto de elementos segundo uma relação de ordem, como a ordem crescente de números. Trata-se de uma das operações mais frequentes da computação, pois muitas outras dependem dela: a busca binária só funciona em dados ordenados, bancos de dados ordenam registros para montar índices e relatórios, e listas classificadas aparecem em praticamente todo sistema (KNUTH, 1998). Para algoritmos que ordenam apenas comparando pares de elementos, o melhor desempenho possível no caso geral é proporcional a n log n comparações, em que n é a quantidade de elementos (CORMEN et al., 2012). O Mergesort e o

Quicksort atingem essa ordem de grandeza no caso médio, mas por caminhos diferentes, o que justifica compará-los na prática.

1.1 Mergesort bottom-up

O Mergesort segue a estratégia de divisão e conquista e tem como operação central a intercalação (*merge*): dados dois blocos já ordenados, compara-se o primeiro elemento de cada um, copia-se o menor para um vetor auxiliar e avança-se naquele bloco, até que os dois se esgotem (CORMEN et al., 2012). Na versão recursiva tradicional, chamada *top-down*, o vetor é dividido ao meio repetidamente até restarem blocos de um elemento, que são intercalados na volta da recursão. A versão *bottom-up*, estudada neste trabalho, dispensa a divisão e a recursão: parte do fato de que cada elemento isolado já é um bloco ordenado e intercala blocos vizinhos de largura 1, depois 2, 4, 8 e assim por diante, dobrando a largura a cada passada até que um único bloco cubra todo o vetor (SEDGEWICK; WAYNE, 2011).

Como a largura dobra a cada passada, são necessárias $\log_2 n$ passadas (arredondado para cima), e cada uma percorre os n elementos, o que resulta em tempo O(n log n) no melhor, no médio e no pior caso. Entre os pontos positivos estão o tempo garantido, independente da disposição dos dados; a estabilidade, isto é, elementos iguais mantêm a ordem relativa original; e, na versão *bottom-up*, a ausência de recursão, o que elimina o risco de estouro de pilha e facilita o uso em listas ligadas e na ordenação de arquivos grandes. Entre os pontos negativos estão o consumo de memória auxiliar proporcional à entrada, O(n), e o custo das cópias entre o vetor original e o auxiliar, que aumenta o tempo constante de cada passada.

1.2 Quicksort clássico

O Quicksort, proposto por Hoare (1962), também usa divisão e conquista, mas faz o trabalho pesado antes da recursão. Escolhe-se um elemento chamado pivô e o vetor é particionado: elementos menores que o pivô ficam à esquerda e maiores à direita. Em seguida, cada parte é ordenada recursivamente. Na versão clássica utilizada neste trabalho, o pivô é o elemento central do trecho e dois índices percorrem o vetor em sentidos opostos, trocando os pares de elementos que estão do lado errado até se cruzarem (ZIVIANI, 2011).

Seus pontos positivos são a ordenação no próprio vetor (*in-place*), sem vetor auxiliar, com memória extra apenas para a pilha de recursão, tipicamente O(log n), e o excelente desempenho prático no caso médio, O(n log n), favorecido pelo acesso sequencial à memória e pelo pequeno número de movimentações. Os pontos negativos são o pior caso $O(n^2)$, que ocorre quando o pivô escolhido é repetidamente o menor ou o maior elemento do trecho; a falta de

estabilidade; e a dependência da recursão. Por isso, a escolha do pivô é decisiva para o desempenho do algoritmo (SEDGEWICK; WAYNE, 2011).

1.3 Objetivo

Nosso objetivo é comparar experimentalmente o tempo de execução do Mergesort *bottom-up* e do Quicksort clássico em conjuntos de 30 a 300.000 números, analisando os resultados por meio de estatística descritiva, *boxplots* e gráficos de barras, e relacionando o comportamento observado com a teoria de cada algoritmo.

2 MATERIAL E MÉTODOS

2.1 Ferramentas e ambiente

Implementamos os dois algoritmos na linguagem C++ (padrão C++11), em um único programa de medição. O projeto foi preparado para ser compilado e executado no Windows 11 com a IDE Embarcadero Dev-C++: o script PAINEL_ADM.bat localiza o compilador g++ (TDM-GCC) da IDE, compila o código e executa os testes por meio de um menu. Para as medições apresentadas neste artigo, usamos um servidor Linux de terceiros, acessado remotamente, com processador Intel Xeon 2,10 GHz (4 núcleos), 15 GB de RAM e sistema Linux (kernel 6.18), com o compilador g++ 13.3 (GCC), padrão C++11, otimização -O2. Gravamos os tempos de cada amostra em arquivos CSV e os organizamos em uma planilha do Microsoft Excel (planilha_resultados.xlsx, que acompanha este trabalho), na qual calculamos a estatística descritiva com as funções MÉDIA, MED, DESVPAD, VAR, MÁXIMO e MÍNIMO. Os gráficos foram feitos em Python 3 com a biblioteca Matplotlib, e conferimos os valores da planilha com os calculados em Python e pelo próprio programa em C++, sem nenhuma divergência.

2.2 Implementação

No Mergesort *bottom-up*, usamos dois laços: o externo controla a largura dos blocos (1, 2, 4, 8, ...) e o interno percorre o vetor intercalando cada par de blocos vizinhos com o auxílio de um vetor auxiliar do mesmo tamanho. Quando o tamanho do vetor não é potência de 2, o último bloco é menor ou fica sem par e é intercalado em uma passada posterior. Já o Quicksort clássico ficou recursivo, com pivô igual ao elemento central do trecho e particionamento por dois índices que avançam um em direção ao outro, trocando os elementos fora de lugar.

2.3 Procedimento experimental

Seguimos os passos abaixo:

1) Definimos os cinco tamanhos de conjunto exigidos: 30, 300, 3.000, 30.000 e 300.000 números inteiros.
2) Para cada tamanho, coletamos 20 amostras de tempo de cada algoritmo, totalizando 200 ordenações.
3) Geramos os números aleatoriamente entre 0 e 1.000.000 com o gerador Mersenne Twister (std::mt19937), com semente fixa 42, o que permite reproduzir exatamente os mesmos dados.
4) Em cada amostra, os dois algoritmos ordenaram cópias dos mesmos vetores, garantindo uma comparação justa.
5) Antes das amostras de cada tamanho, fizemos uma execução de aquecimento, descartada, para reduzir efeitos de memória cache e de alocação inicial.
6) Medimos o tempo com o relógio monotônico std::chrono::steady_clock, envolvendo apenas a chamada de ordenação; a geração e a cópia dos dados ficaram fora da medição.
7) Como vetores pequenos são ordenados em frações de microssegundo, abaixo da precisão confiável do relógio, cada amostra ordenou k vetores aleatórios diferentes ($k = 20.000 / n$; $k = 1$ para $n \geq 20.000$) e registrou o tempo médio por vetor.
8) Após cada ordenação, verificamos o resultado com std::is_sorted; não houve nenhuma falha.
9) Gravamos os tempos em CSV e os organizamos na planilha do Excel, onde calculamos a estatística descritiva; em seguida, elaboramos os *boxplots* e os gráficos de barras.

Um cuidado que só percebemos durante os testes foi a forma de medir os vetores pequenos. Em uma versão preliminar, repetíamos k vezes o mesmo vetor, e os tempos de 30 e 300 elementos ficaram artificialmente baixos: o crescimento de 300 para 3.000 elementos passou de 40 vezes, muito acima das cerca de 14 vezes previstas pela teoria. Isso acontece porque, ao ordenar repetidamente a mesma entrada, o processador aprende o padrão das comparações (previsão de desvios) e executa mais rápido do que faria com dados novos. Por isso, passamos a usar k vetores diferentes em cada amostra, o que trouxe os resultados para perto da previsão teórica, como mostra a Tabela 6.

2.4 Estatística descritiva

Para cada algoritmo e tamanho, calculamos: a média aritmética; a mediana, valor central das 20 amostras ordenadas; o desvio padrão amostral, com divisor n − 1; a variância amostral, quadrado do desvio padrão, expressa em ms^2; o coeficiente de variação, dado por CV = (desvio padrão ÷ média) × 100, que mede a dispersão relativa em porcentagem; e os valores máximo e mínimo. No Excel, o desvio padrão e a variância amostrais correspondem às funções DESVPAD e VAR. Nos *boxplots*, a caixa vai do primeiro ao terceiro quartil, a linha interna marca a mediana e os pontos isolados indicam valores atípicos. Neste trabalho, consideramos que há diferença consistente entre os algoritmos quando as caixas (intervalos interquartis) não se sobrepõem.

3 RESULTADOS E DISCUSSÃO

As Tabelas 1 a 5 apresentam a estatística descritiva dos tempos de execução para cada tamanho de conjunto. A Tabela 6 resume o crescimento do tempo médio entre tamanhos consecutivos, e as Figuras 1 a 4 ilustram a distribuição das amostras, as médias, a variabilidade relativa e o comportamento em outros tipos de entrada. Os dados brutos das 200 amostras e as fórmulas utilizadas estão na planilha que acompanha o trabalho, nas abas Dados, Estatística e Crescimento.

A Tabela 1 ilustra a estatística descritiva dos dados obtidos com 30 números.

Tabela 1. Estatística descritiva dos algoritmos de ordenação com o conjunto de 30 dados.

	Variáveis					
Algoritmo	**Média (ms)**	**Mediana (ms)**	**Desvio Padrão (ms)**	**Variância (ms^2)**	**Coeficiente de Variação (%)**	**Máximo / Mínimo (ms)**
Mergesort bottom-up	0,000752	0,000749	0,0000287	$8{,}24 \times 10^{-10}$	3,8	0,000809 / 0,000709
Quicksort clássico	0,000789	0,000780	0,0000418	$1{,}74 \times 10^{-9}$	5,3	0,000899 / 0,000734

Fonte: Autores (2026).

Conforme a Tabela 1, com o conjunto de 30 dados os dois algoritmos ordenaram cada vetor em menos de um microssegundo: média de 0,000752 ms para o Mergesort *bottom-up* e 0,000789 ms para o Quicksort clássico. A média do Mergesort foi 4,8% menor, mas essa diferença é da mesma ordem dos coeficientes de variação (3,8% e 5,3%) e as caixas do *boxplot* se sobrepõem (Figura 1), o que caracteriza empate técnico. Com tão poucos elementos, o custo é

dominado por operações fixas, como a chamada da função e a alocação do vetor auxiliar do Mergesort, e não pelo número de comparações.

A Tabela 2 ilustra a estatística descritiva dos dados obtidos com 300 números.

Tabela 2. Estatística descritiva dos algoritmos de ordenação com o conjunto de 300 dados.

	Variáveis					
Algoritmo	**Média (ms)**	**Mediana (ms)**	**Desvio Padrão (ms)**	**Variância (ms^2)**	**Coeficiente de Variação (%)**	**Máximo / Mínimo (ms)**
Mergesort bottom-up	0,0116	0,0116	0,000323	$1,04 \times 10^{-7}$	2,8	0,0123 / 0,0110
Quicksort clássico	0,0120	0,0118	0,000754	$5,69 \times 10^{-7}$	6,3	0,0146 / 0,0110

Fonte: Autores (2026).

A Tabela 2 mostra o mesmo comportamento para 300 dados: 0,0116 ms em média para o Mergesort *bottom-up* e 0,0120 ms para o Quicksort clássico, diferença de 3,6%. As medianas (0,0116 ms e 0,0118 ms) confirmam a proximidade, e a baixa dispersão (CV de 2,8% e 6,3%) indica medições estáveis. Também neste tamanho não há diferença consistente entre os algoritmos.

A Tabela 3 ilustra a estatística descritiva dos dados obtidos com 3.000 números.

Tabela 3. Estatística descritiva dos algoritmos de ordenação com o conjunto de 3.000 dados.

	Variáveis					
Algoritmo	**Média (ms)**	**Mediana (ms)**	**Desvio Padrão (ms)**	**Variância (ms^2)**	**Coeficiente de Variação (%)**	**Máximo / Mínimo (ms)**
Mergesort bottom-up	0,157	0,156	0,00230	$5,30 \times 10^{-6}$	1,5	0,160 / 0,153
Quicksort clássico	0,155	0,151	0,00824	$6,78 \times 10^{-5}$	5,3	0,182 / 0,146

Fonte: Autores (2026).

Para 3.000 dados (Tabela 3), as médias foram 0,157 ms (Mergesort *bottom-up*) e 0,155 ms (Quicksort clássico), uma diferença de apenas 1,4%. É o ponto em que o Quicksort começa a se aproximar e a ultrapassar o Mergesort, mas as caixas ainda se sobrepõem. O Quicksort apresentou maior variabilidade (CV de 5,3% contra 1,5%), coerente com o fato de que a

qualidade de cada partição depende dos valores sorteados, enquanto o Mergesort executa sempre a mesma sequência de passadas.

A Tabela 4 ilustra a estatística descritiva dos dados obtidos com 30.000 números.

Tabela 4. Estatística descritiva dos algoritmos de ordenação com o conjunto de 30.000 dados.

	Variáveis					
Algoritmo	**Média (ms)**	**Mediana (ms)**	**Desvio Padrão (ms)**	**Variância (ms^2)**	**Coeficiente de Variação (%)**	**Máximo / Mínimo (ms)**
Mergesort bottom-up	2,26	2,26	0,203	0,0413	9,0	3,00 / 1,97
Quicksort clássico	2,06	2,04	0,164	0,0269	8,0	2,65 / 1,83

Fonte: Autores (2026).

Com 30.000 dados (Tabela 4) surge uma diferença consistente: o Quicksort clássico teve média de 2,06 ms e mediana de 2,04 ms, contra 2,26 ms e 2,26 ms do Mergesort *bottom-up*, ou seja, foi 8,9% mais rápido na média. Os intervalos interquartis não se sobrepõem: 75% das amostras do Quicksort ficaram abaixo de 2,07 ms, enquanto 75% das amostras do Mergesort ficaram acima de 2,20 ms. A partir deste tamanho, o vetor de trabalho deixa de caber nos níveis mais rápidos da memória cache, e passa a pesar o fato de o Mergesort copiar todos os elementos para o vetor auxiliar e de volta em cada passada, enquanto o Quicksort apenas troca elementos no próprio vetor.

A Tabela 5 ilustra a estatística descritiva dos dados obtidos com 300.000 números.

Tabela 5. Estatística descritiva dos algoritmos de ordenação com o conjunto de 300.000 dados.

	Variáveis					
Algoritmo	**Média (ms)**	**Mediana (ms)**	**Desvio Padrão (ms)**	**Variância (ms^2)**	**Coeficiente de Variação (%)**	**Máximo / Mínimo (ms)**
Mergesort bottom-up	27,28	28,41	2,99	8,92	10,9	30,79 / 22,26
Quicksort clássico	23,14	24,40	2,89	8,34	12,5	26,76 / 18,54

Fonte: Autores (2026).

A Tabela 5 apresenta o maior conjunto, com 300.000 dados. O Quicksort clássico manteve a vantagem, com média de 23,14 ms e mediana de 24,40 ms, contra 27,28 ms e 28,41 ms do Mergesort *bottom-up*: tempos 15,2% menores na média e 14,1% menores na mediana. Neste tamanho a dispersão foi maior para os dois algoritmos (CV de 10,9% e 12,5%), pois cada ordenação dura dezenas de milissegundos e fica mais exposta a interferências do sistema operacional, como outros processos e o escalonamento do processador. Por isso, as caixas se sobrepõem parcialmente, mas tanto a média quanto a mediana e o mínimo favorecem o Quicksort. Execuções adicionais de verificação mantiveram o mesmo padrão.

A Tabela 6 ilustra quantas vezes o tempo médio cresceu a cada vez que o conjunto de dados foi multiplicado por 10, comparado com o crescimento previsto pela teoria.

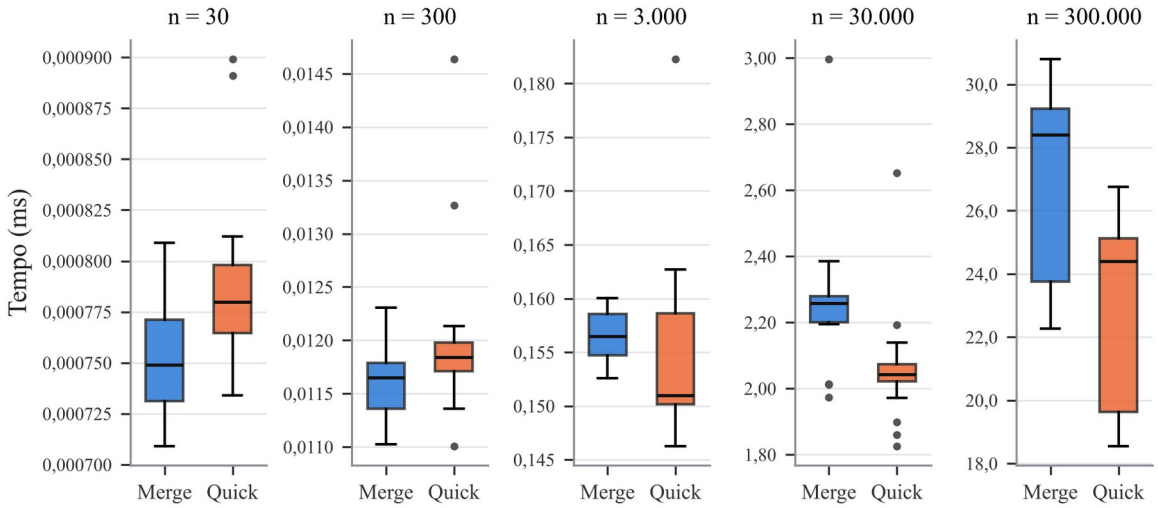
Tabela 6. Crescimento do tempo médio a cada vez que o conjunto de dados é multiplicado por 10.

Tamanhos	**Previsto por n log n**	**Mergesort bottom-up**	**Quicksort clássico**
30 → 300	16,8×	15,4×	15,2×
300 → 3.000	14,0×	13,5×	12,9×
3.000 → 30.000	12,9×	14,4×	13,3×
30.000 → 300.000	12,2×	12,1×	11,2×

Fonte: Autores (2026).

A Tabela 6 relaciona os resultados com a teoria. Se o tempo fosse proporcional a n log n, multiplicar a entrada por 10 deveria multiplicar o tempo pelos valores da coluna "Previsto", entre 12,2 e 16,8 vezes. Os valores medidos ficaram entre 11,2 e 15,4 vezes, muito próximos do previsto, o que confirma experimentalmente o comportamento O(n log n) dos dois algoritmos com dados aleatórios. Um algoritmo quadrático, como o Insertionsort no caso médio, teria o tempo multiplicado por cerca de 100 a cada salto, o que tornaria inviável ordenar 300.000 elementos nas mesmas condições.

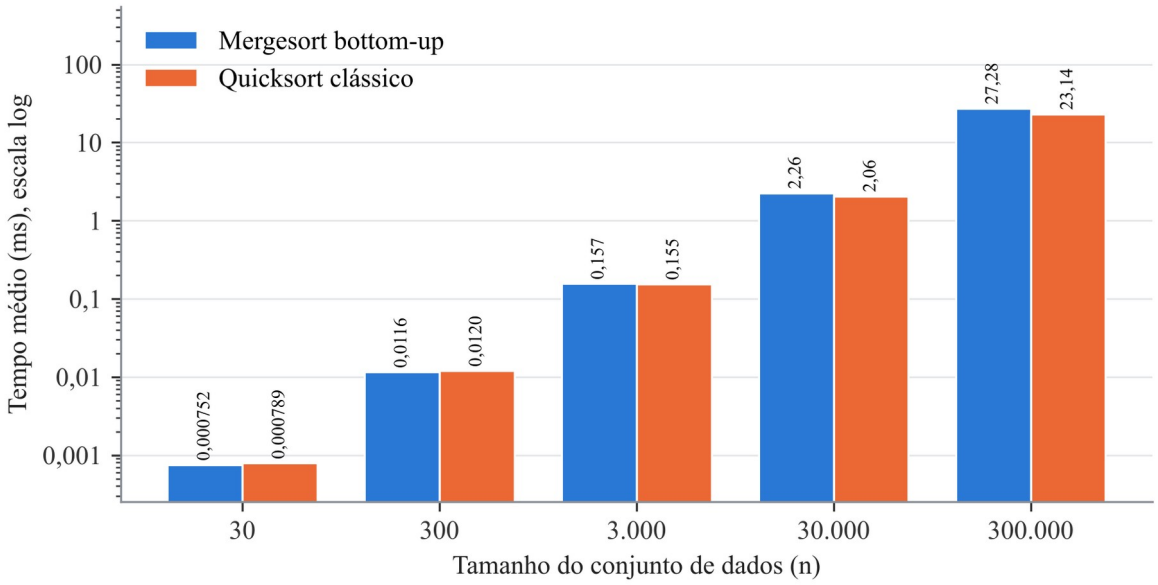
Figura 1. *Boxplot* dos tempos de execução das 20 amostras de cada algoritmo, por tamanho do conjunto de dados.



Fonte: Autores (2026).

A Figura 1 mostra a distribuição completa das 20 amostras. Para 30, 300 e 3.000 dados as caixas dos dois algoritmos se sobrepõem, confirmando o empate técnico discutido nas Tabelas 1 a 3. Em 30.000 dados as caixas ficam separadas, com o Quicksort clássico claramente abaixo, e em 300.000 a mediana e a caixa do Quicksort continuam deslocadas para tempos menores. Os pontos isolados são amostras atípicas, em geral mais lentas, provocadas por interferências momentâneas do sistema; por isso a mediana, menos sensível a esses valores, é uma referência importante ao lado da média.

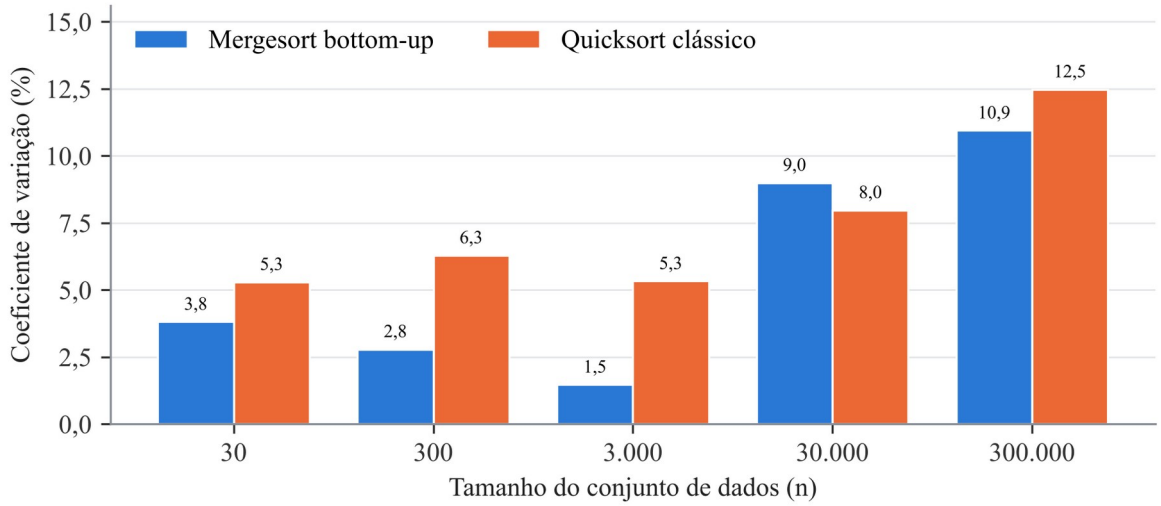
Figura 2. Tempo médio de execução por tamanho do conjunto de dados (escala logarítmica).



Fonte: Autores (2026).

Na Figura 2, as médias aparecem em escala logarítmica, necessária porque os tempos variam de frações de microssegundo a dezenas de milissegundos. As barras crescem em degraus quase iguais a cada tamanho, reflexo do crescimento n log n visto na Tabela 6. Nos três menores tamanhos as barras dos dois algoritmos praticamente se igualam, e nos dois maiores a barra do Quicksort clássico fica visivelmente abaixo da do Mergesort *bottom-up*.

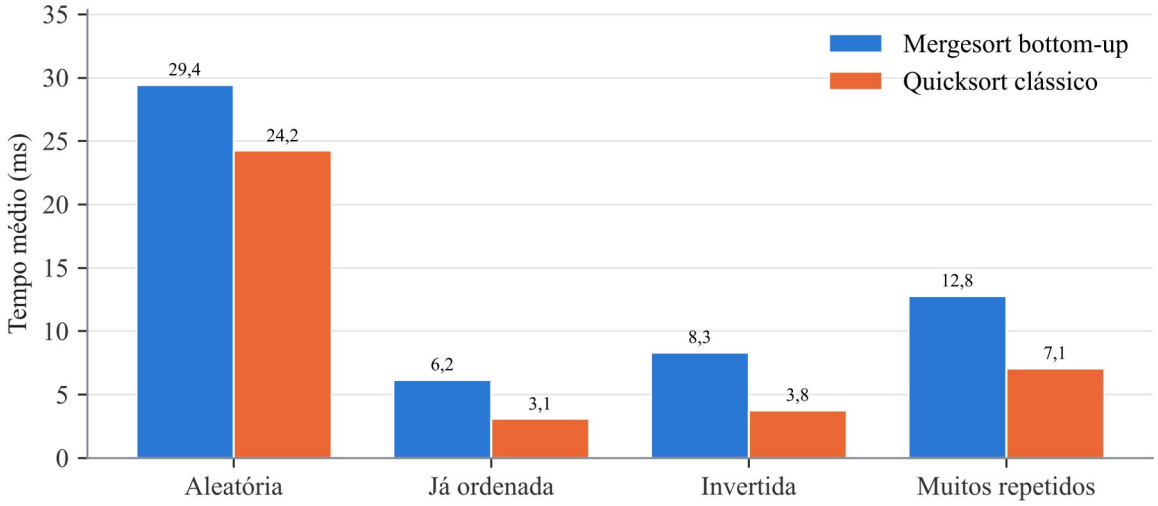
Figura 3. Coeficiente de variação dos tempos de execução por tamanho do conjunto de dados.



Fonte: Autores (2026).

A Figura 3 compara a variabilidade relativa das medições. Todos os coeficientes de variação ficaram entre 1,5% e 12,5%, valores baixos a moderados, o que indica que 20 amostras foram suficientes para caracterizar cada caso. Nos tamanhos pequenos o Quicksort variou mais que o Mergesort, pois o equilíbrio das partições depende dos valores sorteados, enquanto o Mergesort repete sempre o mesmo padrão de intercalações. Nos tamanhos maiores a variação dos dois aumenta e se aproxima, porque passa a predominar a interferência do ambiente de execução sobre ordenações mais longas.

Figura 4. Tempo médio com 300.000 dados em diferentes tipos de entrada.



Fonte: Autores (2026).

Como complemento, a Figura 4 apresenta testes com 300.000 dados em quatro tipos de entrada, com as mesmas 20 amostras por caso. Com dados aleatórios repete-se a vantagem de 17,7% do Quicksort. Com dados já ordenados ou em ordem inversa, os dois algoritmos ficaram bem mais rápidos, e a vantagem do Quicksort clássico subiu para 50,1% e 54,7%: nesses casos o elemento central é exatamente a mediana do trecho, o pivô ideal, e quase nenhuma troca é necessária. Esse resultado mostra que a escolha do pivô central evita o pior caso $O(n^2)$ em entradas ordenadas, que ocorreria se o pivô fosse o primeiro ou o último elemento. Com muitos valores repetidos (apenas 10 valores distintos) o Quicksort também foi 44,6% mais rápido, pois o particionamento por dois índices distribui os elementos iguais ao pivô entre os dois lados.

4 CONCLUSÃO

Comparamos o Mergesort *bottom-up* e o Quicksort clássico em 200 ordenações de 30 a 300.000 números, com a mesma entrada para os dois algoritmos e tratamento por estatística descritiva. Para conjuntos de até 3.000 elementos os algoritmos empataram tecnicamente, com diferenças menores que a variação natural das medições. A partir de 30.000 elementos o Quicksort clássico foi consistentemente mais rápido, com tempo médio 8,9% menor em 30.000 dados e 15,2% menor em 300.000 dados, vantagem que se manteve e até aumentou em entradas ordenadas, invertidas e com valores repetidos.

O crescimento do tempo dos dois algoritmos acompanhou de perto a previsão O(n log n), confirmando na prática a análise teórica. A diferença entre eles, portanto, não está na ordem de complexidade, mas nas constantes: o Mergesort copia todos os elementos para um vetor

auxiliar a cada passada, enquanto o Quicksort reorganiza o próprio vetor. Para o grupo, esse foi o principal aprendizado: a notação O mostra como o tempo cresce, mas não decide sozinha qual algoritmo é mais rápido. Detalhes de implementação, como as cópias de memória e a escolha do pivô, fazem diferença real, e a própria forma de medir precisa de cuidado para não produzir números enganosos, como vimos com os vetores pequenos.

Concluímos que o Quicksort clássico é a escolha mais indicada quando o objetivo é o menor tempo médio com pouco uso de memória, desde que o pivô seja bem escolhido, como o elemento central usado neste trabalho. Já o Mergesort *bottom-up* é preferível quando é necessário garantir tempo O(n log n) mesmo no pior caso, manter a estabilidade da ordenação ou evitar recursão. Como trabalho futuro, pretendemos repetir o experimento em outros computadores e incluir algoritmos quadráticos, como o Insertionsort, para evidenciar a diferença entre as classes de complexidade.

REFERÊNCIAS

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. **Algoritmos**: teoria e prática. 3. ed. Rio de Janeiro: Elsevier, 2012.

HOARE, C. A. R. Quicksort. **The Computer Journal**, v. 5, n. 1, p. 10-16, 1962.

KNUTH, D. E. **The art of computer programming**: sorting and searching. 2. ed. Reading: Addison-Wesley, 1998. v. 3.

SEDGEWICK, R.; WAYNE, K. **Algorithms**. 4. ed. Boston: Addison-Wesley, 2011.

ZIVIANI, N. **Projeto de algoritmos**: com implementações em Pascal e C. 3. ed. São Paulo: Cengage Learning, 2011.